REDIS · 무엇을 풀려고 나왔나

왜 레디스인가

로컬 캐시는 상태를 **각 서버의 메모리에 묶어둔다.**
레디스는 그 상태를 **서버 밖의 빠른 공용 저장소로 빼낸다.**

쇼핑몰 “정글마켓” 하나로 끝까지 따라가 봅니다

한눈에 보는 흐름

레디스는 “고통”에서 출발했다

1 · 출발점

로컬 캐시는 빠르고 좋은 도구. 서버 한 대면 충분하다

→

2 · 조건

그런데 서버가 여러 대가 된다

→

3 · 문제

그 순간 로컬 캐시가 다섯 가지를 망가뜨린다

→

4 · 뿌리

원인은 하나: 상태가 각 서버에 묶여 있다

→

5 · 해결

상태를 서버 밖 한 곳으로 뺀다 → 레디스

조건은 문제가 터지는 상황일 뿐 원인이 아니다. 같은 코드도 서버가 한 대면 멀쩡하다.

뿌리는 다섯 증상을 하나로 묶는 진짜 원인. 뿌리를 고치면 다섯 개가 한 번에 풀린다.

1 · 출발점

로컬 캐시는 빠르고 좋은 도구다

DB에서 상품 조회

200ms

서버 메모리 캐시에서 조회

0.01ms

```
cache = {} # 서버 메모리 안의 캐시

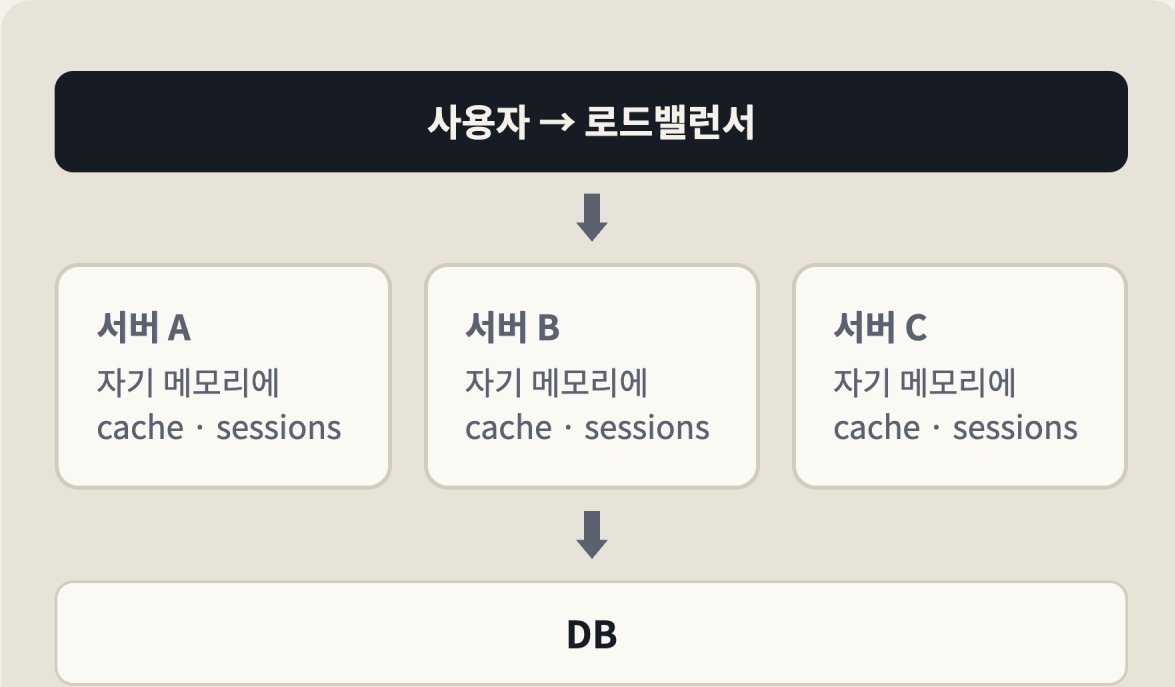
def get_product(pid):
    if pid in cache:
        return cache[pid] # 0.01ms
    product = db.query(pid) # 200ms
    cache[pid] = product
    return product
```

네트워크를 안 타서 가장 빠르고, DB 요청이 줄어 DB를 보호한다.
서버가 한 대라면 여기서 끝. 레디스는 필요 없다.

2 · 조건

서버가 여러 대가 된다

트래픽	세일 날 접속자 10배. 한 대를 키우는 데는 한계가 있어 같은 서버를 여러 대 둔다
장애	한 대가 죽으면 쇼핑몰 전체가 멈춘다. 여러 대면 나머지가 받는다
배포	한 대씩 바꿔야 쇼핑몰을 멈추지 않고 새 버전을 올릴 수 있다



로드밸런서는 요청을 A, B, C에 번갈아 보낸다. **같은 사용자의 요청도 매번 다른 서버로 갈 수 있다.** 코드는 그대로인데, 이제부터 문제가 터진다.

3 · 문제 — 장면 ①

새로고침할 때마다 가격이 바뀐다

관리자가 키보드를 **50,000 → 40,000원**으로 할인. 요청은 서버 A로 간다

서버 A
40,000원
DB 수정 + 자기 캐시 갱신

서버 B
50,000원
모름. 옛 캐시 그대로

서버 C
50,000원
모름. 옛 캐시 그대로

고객이 새로고침할 때마다 A → B → C로 가서 **40,000 → 50,000 → 50,000원**. 결제 화면에선 또 다른 가격

왜?

서버 A는 서버 B, C의 메모리를 지울 방법이 없다

고객 입장

“할인한다더니 가격이 왔다 갔다 하네?” → 문의, 환불 요청

3 · 문제 — 장면 ②

로그인했는데 다음 클릭에서 로그아웃된다

kim이 로그인 → 요청은 서버 A로. A가 세션을 **자기 메모리에** 저장

서버 A

`{abc: kim}`

로그인 기록이 여기에만

서버 B

`{ }`

기록 없음 → “로그인 하세요”

서버 C

`{ }`

기록 없음

kim이 마이페이지 클릭 → 로드밸런서가 서버 B로 → **로그인이 풀린다**

왜?

로그인 기록(세션)이 서버 A의 메모리에만 있다

고객 입장

클릭할 때마다 로그인이 풀렸다 붙었다 한다 → 쇼핑몰을 떠난다

3 · 문제 — 장면 ③

배포할 때마다 DB가 휘청인다

평소 DB 요청 (초당)

100

→

배포 직후 DB 요청 (초당)

5,000

A, B, C 재시작 → 메모리가 날아감 → **캐시 3개가 동시에 텅 빔** → 모든 요청이 DB로

세일 날 서버 D를 추가해도 빈 캐시로 시작 → **오히려 DB 부하가 는다**

왜?

캐시가 서버 프로세스 안에 있어서, 프로세스가 새로 켜지면 같이 비어버린다

결과

캐시의 목적은 DB 보호인데, **가장 위험한 순간에** DB를 지키지 못한다

3 · 문제 — 장면 ④

같은 데이터를 세 번 저장하고, DB에 세 번 묻는다

인기 상품 **1,000개**를 캐시에 올리려면

서버 A
1,000개
DB에 1,000번 조회

서버 B
1,000개
DB에 1,000번 조회
A와 똑같은 내용

서버 C
1,000개
DB에 1,000번 조회
A와 똑같은 내용

서버가 10대면 **메모리 10배, DB 조회 10배**

왜?

같은 데이터를 서버마다 따로 들고, 따로 채운다

한계

캐시 크기는 각 서버의 메모리 한도를 넘을 수 없다

3 · 문제 — 장면 ⑤

함께 세야 하는 값은 아예 만들 수 없다

> 한정판 키보드 **1개** 남음

kim → 서버 A
재고 확인: 1개 → 구매 가능 → 주문

lee → 서버 B
재고 확인: 1개 → 구매 가능 → 주문

→ 1개를 2명에게 팔아버림

만들고 싶은 기능	로컬 캐시로 하면
상품 전체 조회수	서버마다 따로 셈. 합칠 곳이 없음
쿠폰 선착순 100명	서버마다 100명씩, 300명이 받음
결제 버튼 두 번 클릭 방지	다른 서버로 가면 둘 다 결제됨
로그인 5회 실패 시 잠금	서버를 바꿔가며 15번까지 시도

①~④는 “틀리거나 느려지는” 문제, ⑤는 **기능 자체를 만들 수 없는** 문제다.

4 · 뿌리

다섯 장면의 원인은 하나다

상태가 각 서버 프로세스의 메모리에 묶여 있다.

① 가격
서버마다 cache 상태가 다름

② 로그아웃
서버마다 sessions 상태가 다름

③ 배포
재시작하며 상태가 같이 사라짐

④ 중복
같은 상태를 서버마다 따로 들고 있음

⑤ 재고
모두가 봐야 할 상태를 둘 곳이 없음

상태 = 같은 요청인데 답을 다르게 만드는, 서버가 기억하고 있는 데이터

4 · 뿌리 — 상태란

상태가 서버를 stateful로 만든다

```
sessions = {} # ← 이게 상태

mypage("abc")
로그인 전 sessions = {}
  → "로그인하세요"
로그인 후 sessions = {"abc": "kim"}
  → "kim님 마이페이지"
```

요청은 똑같은데 **sessions에 무엇이 있느냐**에 따라 답이 바뀐다.

stateful

자기만 아는 상태를 들고 있음. 요청이 "그 서버"로 가야만 제대로 처리됨

stateless

상태를 안 들고 있음. 어느 서버로 가도 같은 답

구분	서버 메모리의 것	서버가 꺼지면
세션	원본 (그 서버에만)	다시 로그인: 기능이 깨짐
캐시	사본 (원본은 DB)	잠깐 느려짐: 성능만 나빠짐

4 · 뿌리 — 증상만 막으면

하나를 막으면 다른 게 남는다

시도	막는 것	남는 것
캐시 TTL을 짧게 (예: 5초)	① 불일치 시간이 짧아짐	5초 동안은 여전히 다름. 짧을수록 DB 요청이 늘어 ③이 심해짐. ②⑤ 그대로
오래된 키를 지워 메모리 절약	④ 메모리 사용	서버마다 따로 있는 구조는 그대로. ①②③⑤ 그대로
같은 사용자는 같은 서버로 (스티키 세션)	② 로그아웃	그 서버가 죽으면 세션도 사라짐. 부하가 한쪽에 몰림. ①③④⑤ 그대로

증상이 아니라 뿌리를 고쳐야 한다. 상태를 서버에서 떼어낸다.

5 · 해결

상태를 서버 밖 한 곳으로 뺀다



① 가격을 바꾸면 공용 저장소의 키 하나만 지운다 → 모두 새 가격

② 세션이 공용 저장소에 → 어느 서버로 가도 kim

③ 저장소는 앱과 따로 산다 → 재시작해도 캐시 유지

④ 한 벌만 저장하고, DB에도 한 번만 묻는다

⑤ 모두가 같은 재고 키를 본다 → 1개면 한 명만

5 · 해결 — 어디에?

그 공용 저장소를 어디에 둘까

후보 1

앱 서버 안

그게 바로 로컬 캐시. 처음 문제로 돌아간다

✕ 탈락

후보 2

DB

공유는 된다. 하지만 느리고 늘리기 어렵다. 캐시의 목적(DB 보호)과 정반대

△ 부적합

후보 3

메모리 기반 별도 서버

공유되고, RAM이라 빠르고, DB 앞에서 요청을 막아준다

✓ 레디스

리액트가 상태와 **화면**을 분리했듯, 레디스는 상태와 **서버**를 분리한다.

6 · 레디스 이전

Memcached는 공유를 얻고 자료구조를 잃었다

먼저 시도된 것들

시도	한계
스티키 세션	②만 막고 나머지는 그대로
세션 복제	서버가 늘수록 복사 트래픽 폭증
DB에 저장	느리고 DB 부하가 오히려 늘어남
Memcached (2003)	공유는 성공. 하지만 값은 문자열만, 재시작하면 사라짐, 복제 없음

Memcached로 랭킹에 점수 하나 추가

1. 랭킹 **전체**를 문자열로 꺼냄 (GET) · 네트워크 왕복
2. 앱에서 리스트로 바꾸고 점수 수정, 다시 정렬
3. 문자열로 바꿔 **통째로** 저장 (SET) · 네트워크 왕복

데이터가 크면 매번 전부 주고받고, 1~3 사이에 다른 서버가 끼어들면 값이 사라진다.

7 · 레디스 (2009)

공유하면서 자료구조와 운영 기능까지

구체적인 요구	로컬 캐시	Memcached	레디스
모든 서버가 같은 값 보기	× 서버마다 따로	✓	✓
구조 있는 데이터를 일부만 수정	✓ 내 메모리	× 문자열만	✓ List · Hash · ZSet
여러 서버가 동시에 숫자 올리기	×	△ 숫자만	✓ INCR
같은 작업을 한 서버만 하게 (락)	×	△ 흉내만	✓ SET NX EX
재시작해도 데이터 유지	×	×	✓ RDB / AOF
캐시 서버가 죽어도 서비스 유지	해당 없음	× 복제 없음	✓ Replication
데이터가 늘면 나눠 담기	× 메모리 한도	△ 앱이 직접	✓ Cluster
서버끼리 “값이 바뀌었다” 알리기	×	×	✓ Pub/Sub

7 · 레디스 — 동시성

읽고·고치고·쓰기를 레디스 안에서 한 번에

앱에서 나눠 하면 (조회수 100)

A: GET → 100	B: GET → 100
A: SET 101	B: SET 101

→ 두 번 올렸는데 101

레디스 안에서 한 명령으로

```
INCR views:키보드 # → 102
```

- 명령을 한 스레드에서 하나씩 순서대로 처리
- 읽기+더하기+쓰기가 한 명령 → 아무도 못 끼어듦 (원자적)
- 장면 ⑤ 재고: DECR 결과가 0 미만이면 주문 취소

HINCRBY
포인트만 100 올리기

LPUSH + LTRIM
최근 본 상품 10개 유지

SISMEMBER
좋아요 눌렀나 확인

ZINCRBY
판매 랭킹 점수 올리기

8 · 우리 과제

해시 테이블은 모든 자료구조의 입구다

바깥 층 · 키 → 값을 찾는 곳

해시 테이블 하나 ← 우리가 만드는 것

안쪽 층 · 찾은 값의 모양

`String` `List` `Set` `Sorted Set` `Hash`

```
GET price:키보드 → ① 키 찾기 → ② 값 반환
ZINCRBY best 1 키보드 → ① 키 찾기 → ② ZSet 수정
LPUSH recent:kim 키보드 → ① 키 찾기 → ② List 추가
```

왜 하필 해시 테이블인가

리스트	O(N)	처음부터 비교. 키가 늘수록 느려짐
트리	O(log N)	쓰지 않는 정렬에 비용을 냄
해시 테이블	**O(1)**	칸 번호를 바로 계산. “키로 바로 찾기”에 딱

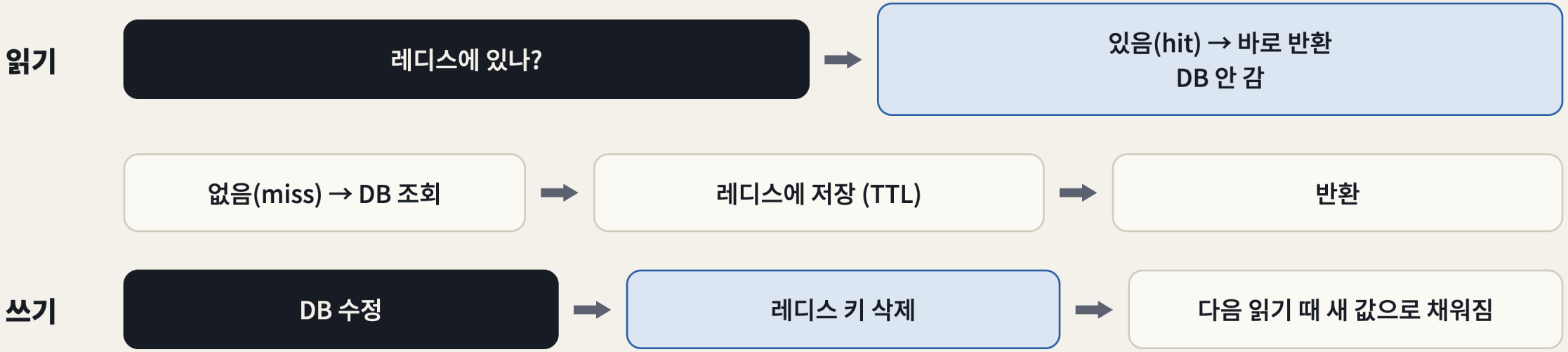
8 · 우리 과제 — 범위

바깥 층과 캐시에 꼭 필요한 기능만 만든다

레디스의 기능	우리 단계
키로 값 찾기 (모든 명령의 입구)	2, 3 · 해시 테이블, 칸 수 확장
EXPIRE 자동 삭제	4 · 만료(지연 삭제) / 13 · 주기적 정리
동시 접근 안전	5 · lock (레디스는 싱글 스레드, 우리는 lock으로 같은 효과)
INCR 원자적 증가	12 · 증가 명령
RDB / AOF 데이터 유지	14 · 데이터 유지 (선택)
DB 앞의 캐시로 쓰기	8 · 데모 API / 9 · 성능 비교
Sorted Set, List, Replication, Cluster	하지 않음 (해시 테이블 위에 얹는 기능)

9 · 실제로는 어떻게 쓰나

Cache-Aside: 레디스를 먼저, 없으면 DB

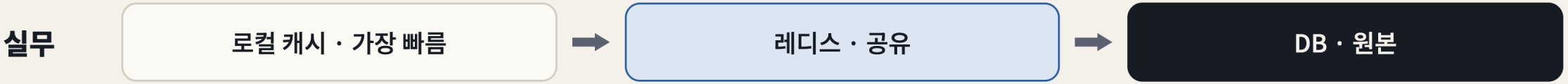


쓰기 때 키를 지우는 것이 장면 ①을 실제로 막는 방법. 원본은 항상 DB, 레디스는 빠른 사본.

10 · 언제

언제 로컬 캐시, 언제 레디스?

데이터 성격	로컬 캐시	레디스
거의 안 바뀜 (카테고리 목록, 설정)	적합	굳이 필요 없음
가끔 바뀌고 잠깐 늦어도 됨 (공지 목록)	TTL로 버틸 수 있음	더 안전함
자주 바뀌고 정확해야 함 (가격, 재고)	부적합	필요
모든 서버가 함께 써야 함 (세션, 락, 카운터)	불가능	필요



레디스가 필요한 기준은 두 가지: **바뀌는 데이터**와 **함께 써야 하는 데이터**.

11 · 비용과 주의점

레디스도 공짜는 아니다

네트워크 비용

로컬은 나노초, 레디스는 약 1ms. 그래도 DB보다 훨씬 빠르다. 그래서 HTTP 대신 가벼운 자체 프로토콜(RESP)을 쓴다

새로운 단일 장애점

모든 서버가 레디스 한 대에 의존한다. Replication, Sentinel이 필요한 이유

DB 대체가 아님

영속성이 있어도 설정에 따라 마지막 몇 초~몇 분은 잃을 수 있다. 원본은 DB에

메모리 한도

RAM에 저장하므로 꽉 차면 정책에 따라 키를 지운다 (예: 오래 안 쓴 키부터, LRU)

정리

상태를 서버 밖으로.

앱 서버가 몇 대든 모두가 같은 상태를 보고, DB는 보호받는다.
레디스는 그 공용 저장소에 **자료구조와 원자적 연산**까지 얹었다.

로컬 캐시: 빠르지만 공유 불가 → Memcached: 공유하지만 문자열만 → 레디스: 공유 + 자료구조 + 운영